ASOCIACIÓN ASTRONÓMICA
Y EDUCATIVA DE CANARIAS
Henrietta Swan Leavitt

UNA VENTANA AL UNIVERSO

2025-2026

-

E5: Introducción a órbitas y agujeros negros

-

Práctica 1: Representando órbitas Newtonianas con Python

David Sola Gil
(AAEC)

1. Introducción

La Ley de Gravitación Universal de Newton constituye un pilar esencial de la física teórica, destacando por su elegante simplicidad y su carácter universal. A pesar del cambio de paradigma con la llegada de la Relatividad General unos 200 años más tarde, sigue capturando, con gran precisión cualitativa y cuantitativa, la dinámica de los cuerpos celestes en nuestro Sistema Solar.

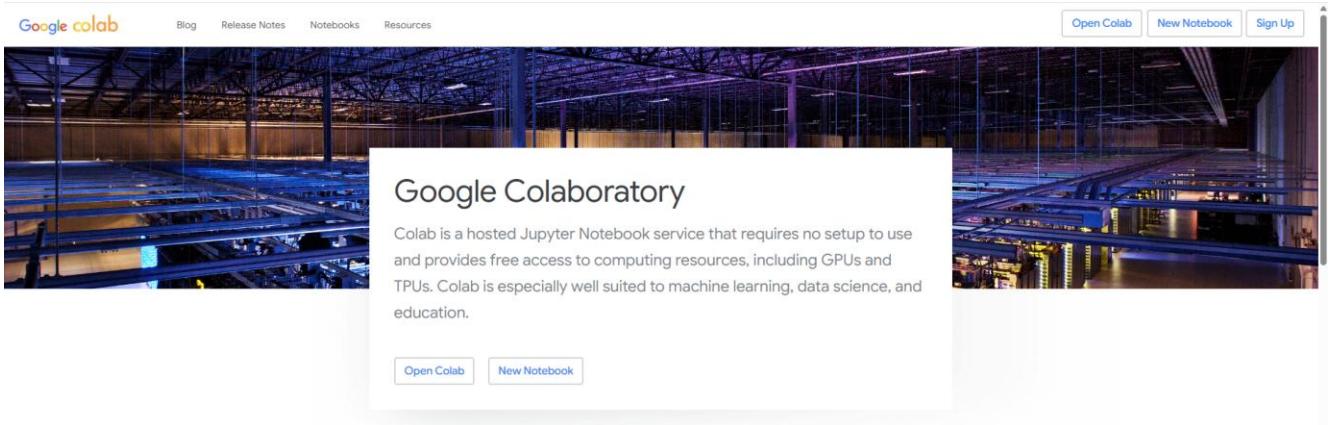
Esta práctica ha sido diseñada para subrayar el valor de la física como herramienta de análisis, tomando como eje central la formulación de Newton. El objetivo principal es mostrar cómo la integración de Python permite enriquecer el proceso de aprendizaje: partiendo de expresiones matemáticas abstractas, el profesorado podrá guiar al alumnado hacia la creación de visualizaciones propias, facilitando así una comprensión más profunda y tangible de los fenómenos gravitatorios.

2. Calentamiento: secciones cónicas

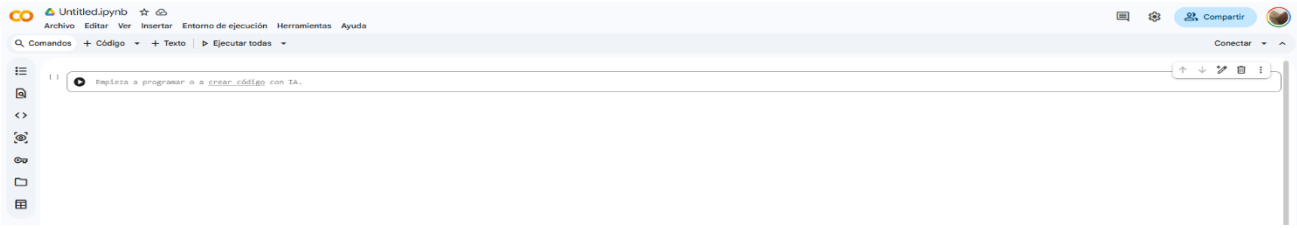
Paso 0: Creando un Jupyter Notebook y primeras gráficas

Antes de comenzar con las representaciones gráficas, conviene que realices un pequeño calentamiento en Python para familiarizarte con las herramientas de código, sus posibilidades y sus limitaciones. Tu punto de partida será Google Colab, una plataforma web que te permite ejecutar código sin necesidad de instalaciones previas. Si tienes la cuenta de Google con la sesión iniciada, puedes guardar los archivos en tu cuenta online. Si no, pueden descargarlos en local para uso futuro. Además, el alumnado puede usarlo en Canarias a partir de los 14 años, con su cuenta de @canariaseducacion.es. Por todas estas ventajas, es un entorno perfecto para llevar la programación al aula.

El primer paso es crear un *Jupyter Notebook*. Para ello, haz click en *New Notebook.*



De aquí, iremos a una página web en la que veremos una barra con opciones y una celda en blanco. En la celda en blanco es donde podremos escribir código en Python.



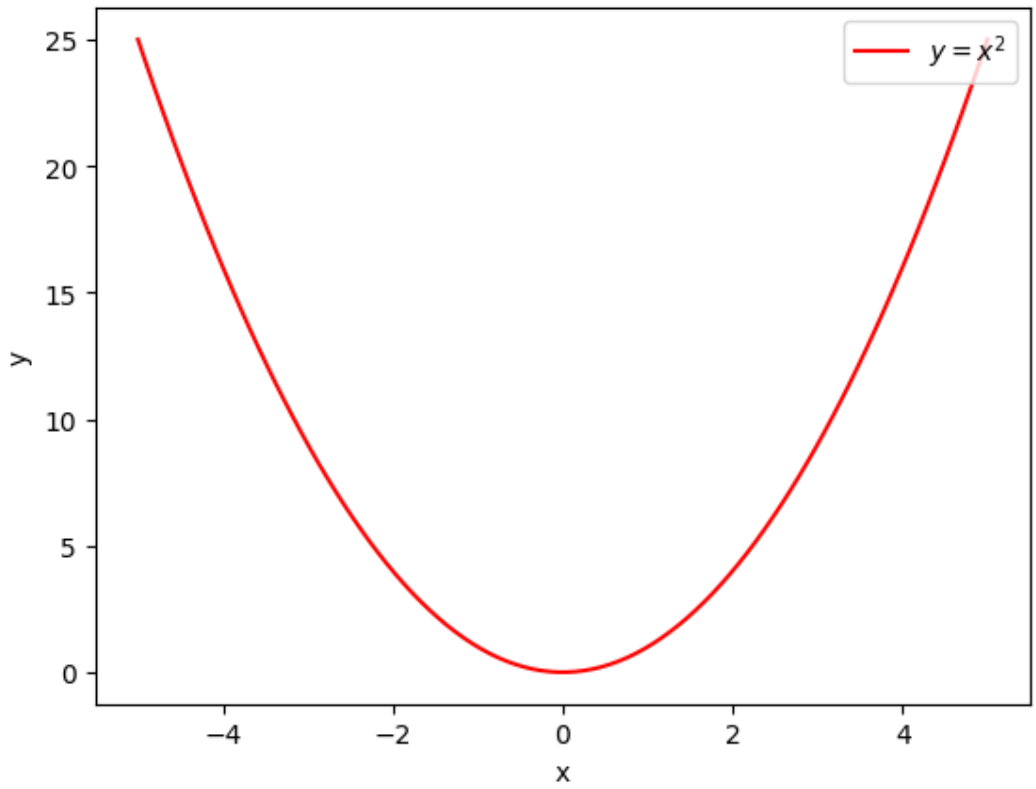
Recuerda que Python utiliza librerías, que son archivos de código con funciones diseñadas para facilitar cálculos y representaciones gráficas profesionales. Dos de las más utilizadas, que cargarás en tu primer paso, son Numpy (para cálculos numéricos) y Matplotlib (para visualización).

```
import numpy as np
import matplotlib.pyplot as plt
```

Aquí estamos cargando las librerías *numpy* y la parte *pyplot* dentro de *matplotlib*, y les estamos dando el alias *np* y *plt* (alias estándar) para cada vez que accedamos a esas librerías, no tener que poner el nombre entero. Nuestro primer ejercicio es, empleando Python, dibujar una gráfica de una parábola. Esto se hace con el código

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-5, 5, 100)
y = x**2
plt.plot(x,y, color='red')
plt.legend(['$y=x^2$'], loc="upper right")
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```



Aquí hemos

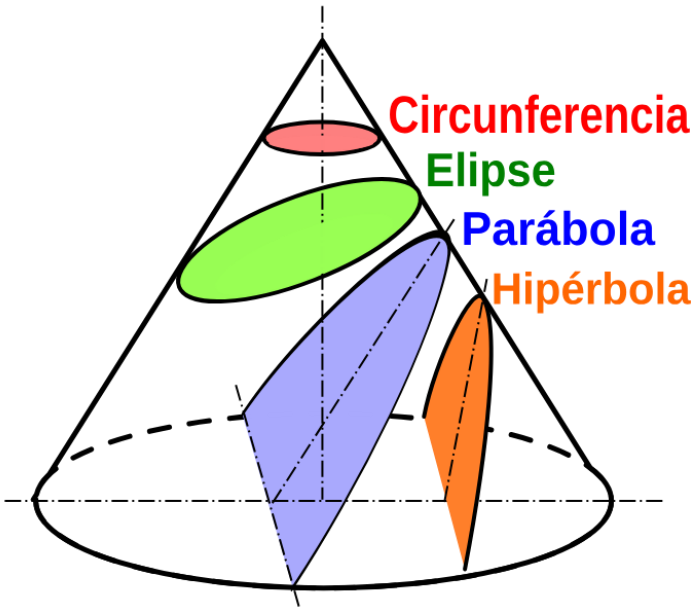
1. Importado las librerías.
2. Utilizado la función np.linspace(-5, 5, 100) para crear un vector de puntos x, que van desde x = -5 a x = 5 en 100 pasos.
3. Elevado cada uno de esos puntos al cuadrado, para obtener una lista de valores y.
4. Creado la gráfica y = x^2 uniendo dichos puntos.
5. Añadido una leyenda al gráfico.
6. Añadido leyendas a ambos ejes.
7. Mostrado la gráfica.

Como primer ejercicio, copia el código de arriba en Python para dibujar una parábola. Una vez generada, te sugiero que modifiques el código para representar la función y = x^3 en el dominio (-10, 10). Experimenta reduciendo el número de puntos o cambiando el color a azul; estas pruebas te ayudarán a familiarizarte con el funcionamiento de la herramienta.

Para una descripción más detallada de todo lo que se puede hacer con Matplotlib, consulta https://aprendeconalf.es/docencia/python/manual/matplotlib/ o la documentación oficial.

Paso 1: Dibujando secciones cónicas

Como vemos en el contenido teórico del módulo, las órbitas de los cuerpos celestes bajo la gravedad de Newton se describen mediante secciones cónicas: circunferencia, elipse, parábola e hipérbola, dibujadas en el gráfico debajo. Para completar este bloque introductorio, te propongo representar estas curvas en el plano (x, y).



Nuestro punto de partida son las ecuaciones ordinarias en función de cada sección cónica.

1. Parábola: $y = 2px^2$, donde p es una constante.
2. Elipse (y la circunferencia como caso especial):

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1,$$

donde a y b son constantes, y la circunferencia corresponde al caso especial $a = b$.

3. Hipérbola:

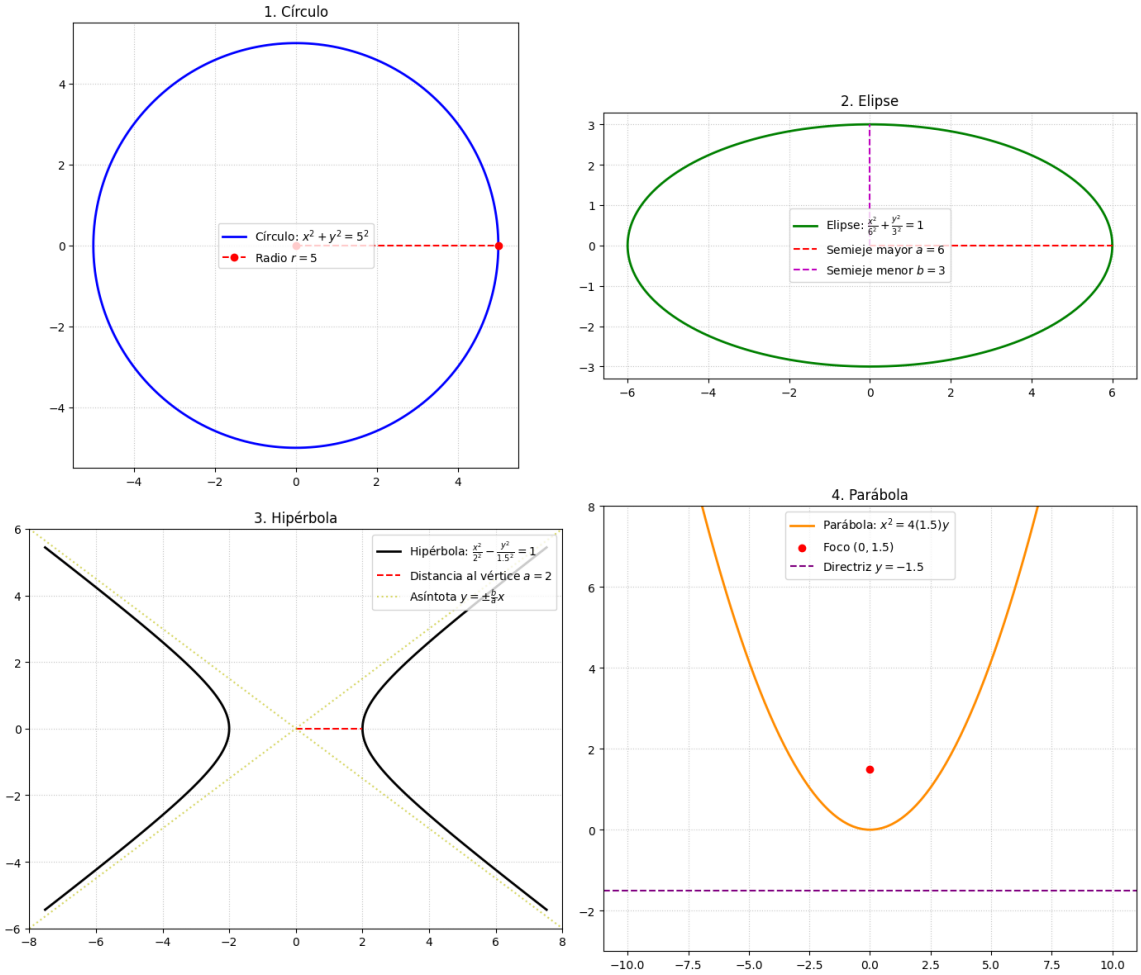
$$\frac{x^2}{a^2} - \frac{y^2}{b^2} = 1,$$

donde a y b son constantes.

Escribe un trocito de código en Python que dibuje un ejemplo de cada una de las secciones cónicas. Aunque puedes intentar dibujarlas mediante "fuerza bruta" dividiendo el plano, te resultará mucho más elegante utilizar su forma paramétrica, expresando (x, y) en términos de un ángulo $\phi \in [0, 2\pi)$. A modo de pista, recuerda las siguientes ecuaciones de funciones trigonométricas:

$$\cos^2(\phi) + \sin^2(\phi) = 1\,,$$
$$\cosh^2(\phi) - \sinh^2(\phi) = 1.$$

El resultado final debería ser similar a la figura debajo.



¿Cuántas constantes necesito para dibujar cada sección cónica? ¿Cómo afecta a cada gráfica el camibio de cada una de las constantes?

3. El potencial efectivo

Tras familiarizarte con las cónicas, inicia tu camino hacia la representación de órbitas. Un paso fundamental antes de resolver cualquier ecuación es intentar extraer la máxima información del problema mediante el análisis.

Con este objetivo, estudiarás la ecuación de la energía E de un cuerpo celeste de masa m, que se encuentra a una distancia r del cuerpo atrayente de masa M. Como explicamos en los apuntes, tenemos que E es constante y viene dada por

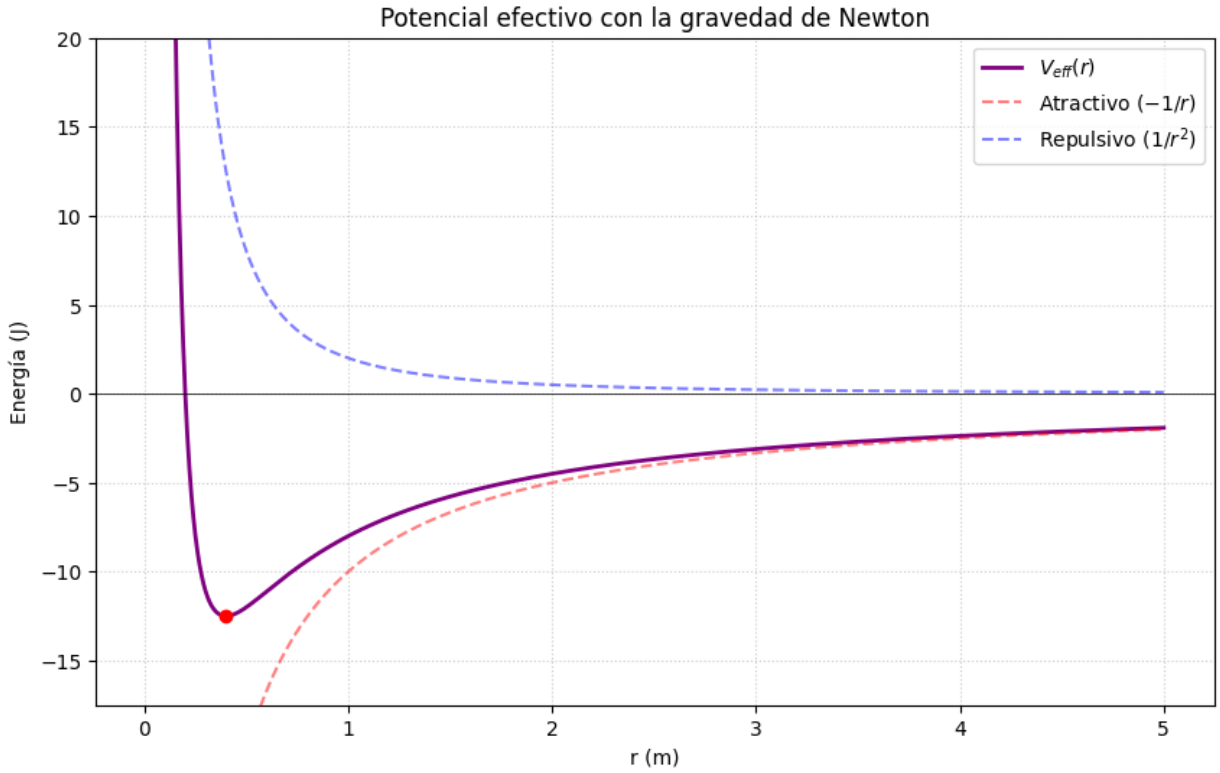
$$E = \frac{1}{2}m\left(\frac{dr}{dt}\right)^2 - \frac{mk}{r} + \frac{ml^2}{2r^2}.$$

Aquí l es el momento angular por unidad de masa, y $k = GM$, donde $G = 6.674 \times 10^{-11}\ N \cdot \frac{m^2}{kg^2}$. Para nuestros propósitos, todas estas variables son constantes. Vamos a representar gráficamente el *potencial efectivo* (todo lo que no es energía cinética), es decir

$$V_{eff}(r) = -\frac{mk}{r} + \frac{ml^2}{2r^2}.$$

Nuestro objetivo ahora es realizar un estudio de esta función. Dibuja, en Python, una gráfica de esta función. Antes escribir código, responde a las siguientes preguntas.

1. ¿Qué ocurre con la función a medida que $r \to 0$? ¿Y para $r \to \infty$?
2. ¿Hay extremos locales? ¿Dónde y de qué tipo?
3. ¿Qué ocurre si $l = 0$? ¿Cuál es la interpretación física de esto?



Para que salga una gráfica bonita, yo he usado las constantes $m = 1, k = 20, l = 2$.

Estudiando la gráfica, vamos a deducir qué tipo de órbita tenemos según la energía E del sistema. A la energía mínima la vamos a llamar E_{min}. Elige la sección cónica que describe la órbita en cada caso.

1. $E = E_{min}$.
2. $E_{min} < E < 0$.
3. $E > 0$.
4. $E = 0$.

Pista: para hacer este ejercicio, es conveniente imaginar líneas horizontales que representan la energía E constante superpuestas a la gráfica anterior. En donde E intersecta V_{eff}, podemos interpretar que la energía cinética es 0. ¿Qué ocurre si entonces incremento la energía cinética un poco?

4. Dibujando órbitas

Una vez comprendidos los parámetros, puedes proceder a la representación de las órbitas. A diferencia de los ejercicios iniciales, observarás que estas secciones cónicas no están centradas en el origen y se expresan de forma más eficiente en coordenadas polares:

$$r(\phi) = \frac{r_0}{e\cos(\phi) + 1},$$

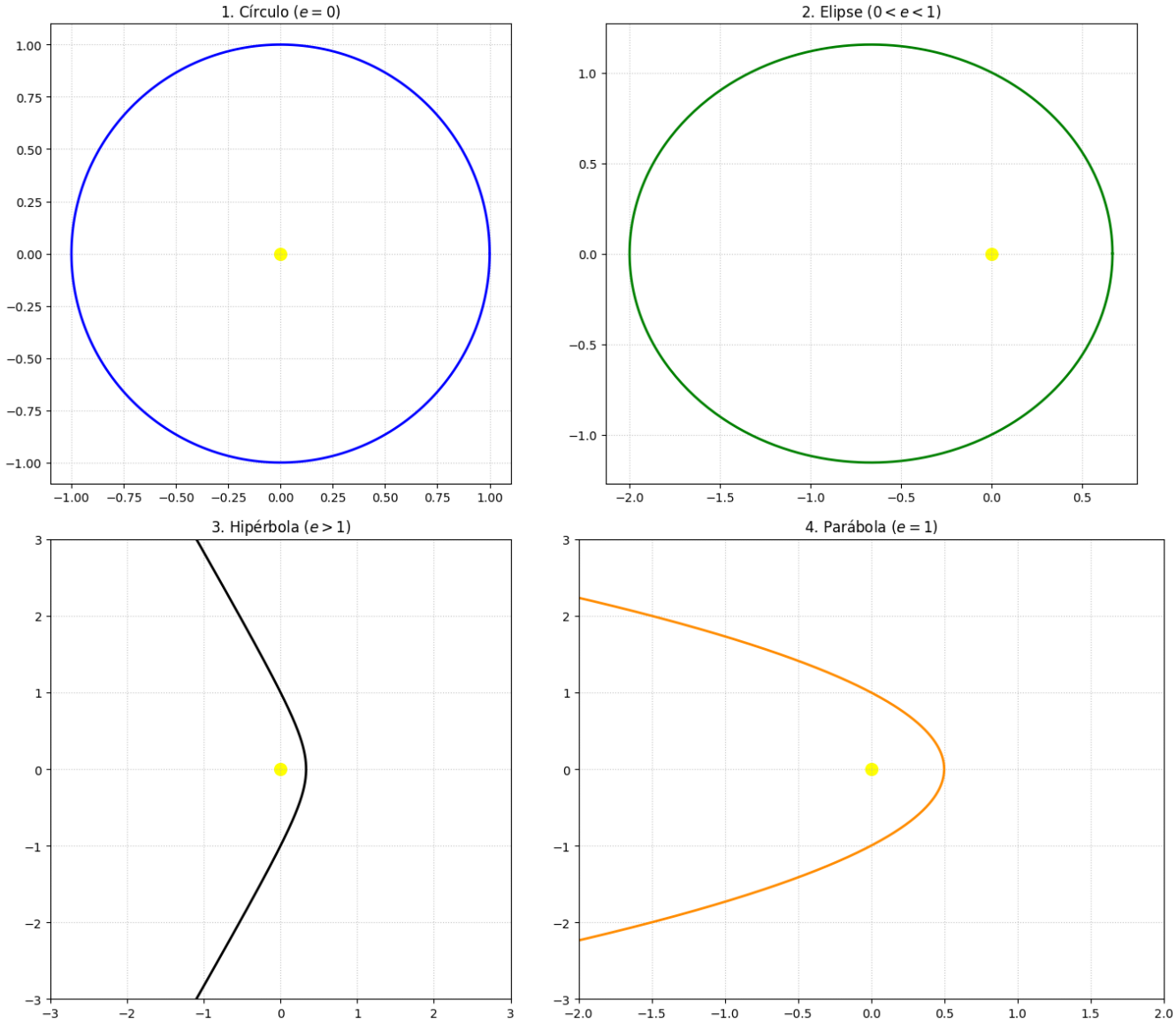
donde $r_0 = l^2/k$ es una constante y $e \in [0, \infty)$ otra constante conocida como la *excentricidad.* A diferencia de la sección 1, estas secciones cónicas no están centradas en el origen, como veremos a continuación. También están expresadas en coordenadas polares, donde

$$x = r\cos(\phi), \quad y = r\sin(\phi).$$

Dibuja las órbitas para $r_0 = 1$ y las excentricidades

1. $e = 0$.
2. $0 < e < 1$.
3. $e = 1$.
4. $e > 1$.

Prueba ahora a incrementar o reducir $r_0 \in (0, \infty)$. ¿Cómo varían las gráficas? ¿En dónde está situado el Sol?



Recuerda que en la sección 2.2 de los apuntes, conectamos la excentricidad y las secciones cónicas con el valor de la energía y las órbitas que producen mediante

$$E = \frac{mk^2}{2l^2}(e^2 - 1)$$

¡Usa esta ecuación como método alternativo para comprobar que las respuestas del ejercicio anterior encajan!

5. Comparando órbitas

Como etapa final, compararás la estimación newtoniana con datos orbitales reales. Aunque estimar órbitas en la realidad es un proceso de gran complejidad que requiere modelos avanzados, verás que la mecánica de Newton captura el movimiento con un margen de error notablemente bajo para muchos casos.

Para estos ejercicios, vamos a emplear datos de la NASA para estimar órbitas planetarias. Estos datos

proceden de una base de datos online conocida como *JPL Horizons,* y tiene información de los planetas del sistema solar, asteroides, satélites e incluso cometas. Gratis. Flipa.

El primer paso aquí es instalar, en nuestra sesión de Google Colab, la librería que nos permite acceder a toda esta información. Simplemente, modificamos el principio de nuestro código para que nos instale la librería correspondiente.

```
!pip install -q astroquery
import numpy as np
import matplotlib.pyplot as plt
from astroquery.jplhorizons import Horizons
```

Tienen ejemplos de cómo cargar la posición y velocidad de planetas utilizando *jplhorizons* en el fichero correspondiente de Python.

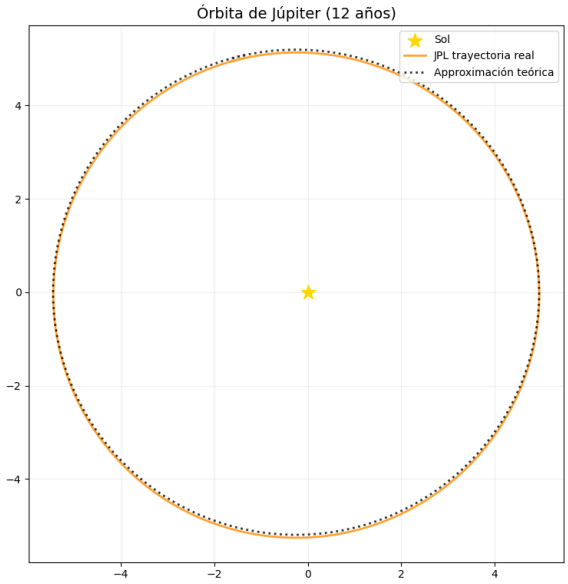
Los datos de *jplhorizons* vienen en formato diccionario, es decir, no son simplemente listas, sino que cada entrada de la lista va asociada a un conjunto de caracteres. En la Jupyter Notebook del Moodle, hay escritas funciones que leen los datos del cuerpo celeste que queramos para extraer los parámetros que podemos poner en la ecuación de la órbita y ahorrar el trabajo de la conversión.

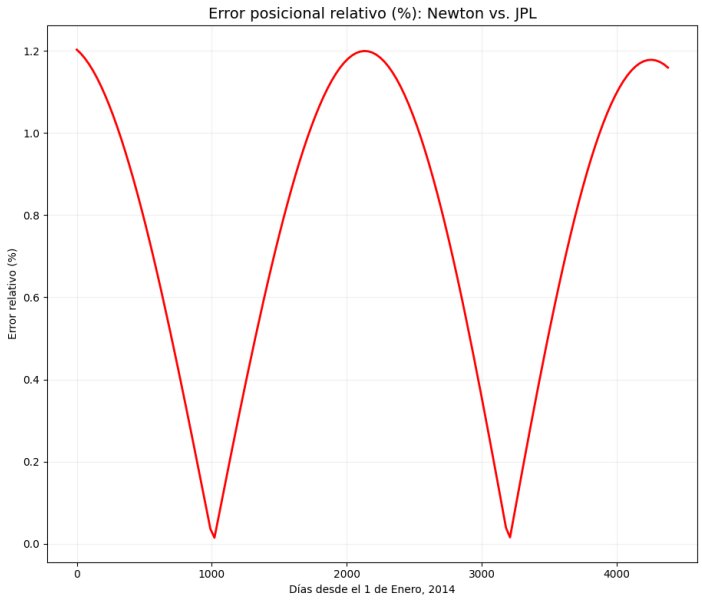
Tras instalar la librería necesaria en tu sesión de Google Colab, te propongo dos retos:

1. Utilizando la posición y la velocidad inicial de, por ejemplo, Júpiter, estima su órbita durante 12 años. Compara el resultado de la aproximación Newtoniana con los datos reales.

2. En 2019, el astrónomo aficionado Gennady Borisov observó el primer cometa procedente de fuera de nuestro sistema solar: 2I/Borisov. Usa los datos de JPL para estimar su trayectoria a través del sistema solar mediante una hipérbola.

A partir de aquí, tu creatividad es el único límite para expandir esta práctica en el aula.





Bibliografía

Imagen secciones cónicas: https://es.wikipedia.org/wiki/Archivo:Secciones_c%C3%B3nicas.svg.